# Databases in Applications

Software Architecture

March 3, 2026

Brae Webb, Evan Hughes & Richard Thomas

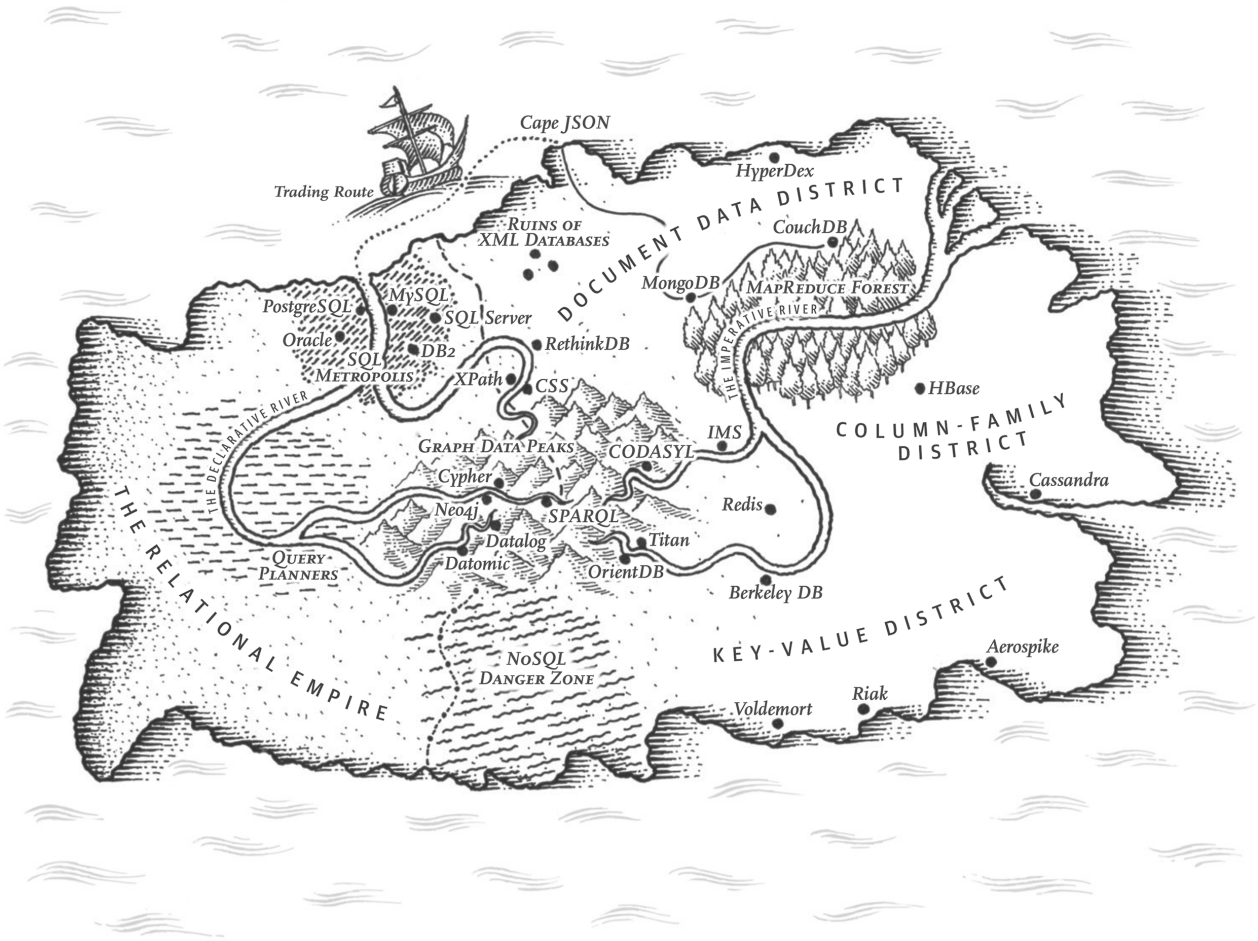


Figure 1: A map of data storage techniques from Designing Data-Intensive Applications [1].

**Aside**

Github Classroom links for this practical can be found on Edstem https://edstem.org/au/courses/33564/discussion/3127162

## 1 This Week

This week our goal is to

- explore the various techniques developers use to store data; and
- upgrade our todo application to use a local SQLite relational database.

# 2 Databases and Data Models

Unfortunately, to build interesting software we often need to store and use data. The storage of data introduces a number of challenges when designing, creating, and maintaining our software. However, not all data storage techniques are created equal; the choice of data storage model can have a profound impact on our software's complexity and maintainability. In this practical, we want to take a superficial exploration of our island of data storage models. For a more in-depth treatment of data storage models that is outside the scope of this course, see Chapter 2 of the *Designing Data-Intensive Applications* book [1].

## 2.1 Relational Storage

Relational databases are what you have been exposed to the most in your university career — think MySQL, Postgres, Oracle DB, etc. This type of database is good at modelling the real world, which is often a highly connected environment.

Some popular offerings are:

- MySQL/MariaDB [ Amazon RDS / Amazon Aurora ]
- Postgres [ Amazon RDS / Amazon Aurora ]
- SQLite

The AWS offerings of these services come in two different types, we have the traditional approach of server capacity ( x cores, y ram ) and we have a server-less approach. The server-less approach is a more dynamic database that can scale to large amounts of load when needed though at a cost per request.

### 2.1.1 ORM

Object Relational Mapping (ORM) is a fairly common tool that programmers use to make developing software with databases easier. One fairly prevalent example of this is SQLAlchemy which is a very widely used database abstraction for Python. SQLAlchemy allows us to move to a higher level of abstraction than SQL queries and perform database actions using standard Python code.

The benefits of ORMs are the ability to model database objects in our existing programming language, instead of having large blocks of SQL text within our source code. The disadvantages come in when we need to do specific SQL work or where the abstraction cost is greater than the benefits.

## 2.2 Wide-Column Storage

Wide-Column databases are a form of NoSQL or non-relational data stores. In these data stores, the data model design focuses on having efficient queries at the cost of data duplication. A warning to the reader is that these models are not flexible after creation, it is much easier to implement a new use case in a relational model.

- Apache Cassandra [ Amazon Keyspaces for Cassandra ]
- Apache HBase

## 2.3 Key-Value Storage

Key-Value stores are very popular for cache or remote configuration use cases, some of the most notable are Redis and Memcached. These stores allow efficient lookup of values via keys and are usually stored in-memory.

- Redis [ Amazon ElastiCache for Redis ]
- Memcached [ Amazon ElastiCache for Memcached]
- Amazon DynamoDB
- Amazon MemoryDB for Redis

## 2.4 Time Series Storage

Time series databases are highly focused storage which is tailored to retrieving results by timestamp ranges. Many implementations also take advantage of the data model to allow efficient rollover of data and partitioning. One of the most popular time series databases is Prometheus, which is used to store monitoring metrics.

- Amazon Timestream.
- TimescaleDB ( Postgres + Addon ).
- Prometheus
- InfluxDB
- PostgreSQL

## 2.5 Document Storage

Document databases are a subset of NoSQL databases, with a focus on a flexible data model. MongoDB for instance allows users to store JSON documents and perform queries on those documents. One advantage of document databases is that they match programmers' existing mental model of storing data in formats such as JSON.

- MongoDB
- Apache CouchDB
- Amazon DocumentDB
- Amazon DynamoDB

## 2.6 Graph Storage

Graph Databases are relational storage with a few enhancements to allow fast neighbour look-ups. These databases also allow the implementation of graph algorithms to query data.

- Amazon Neptune
- Neo4J
- Janus Graph

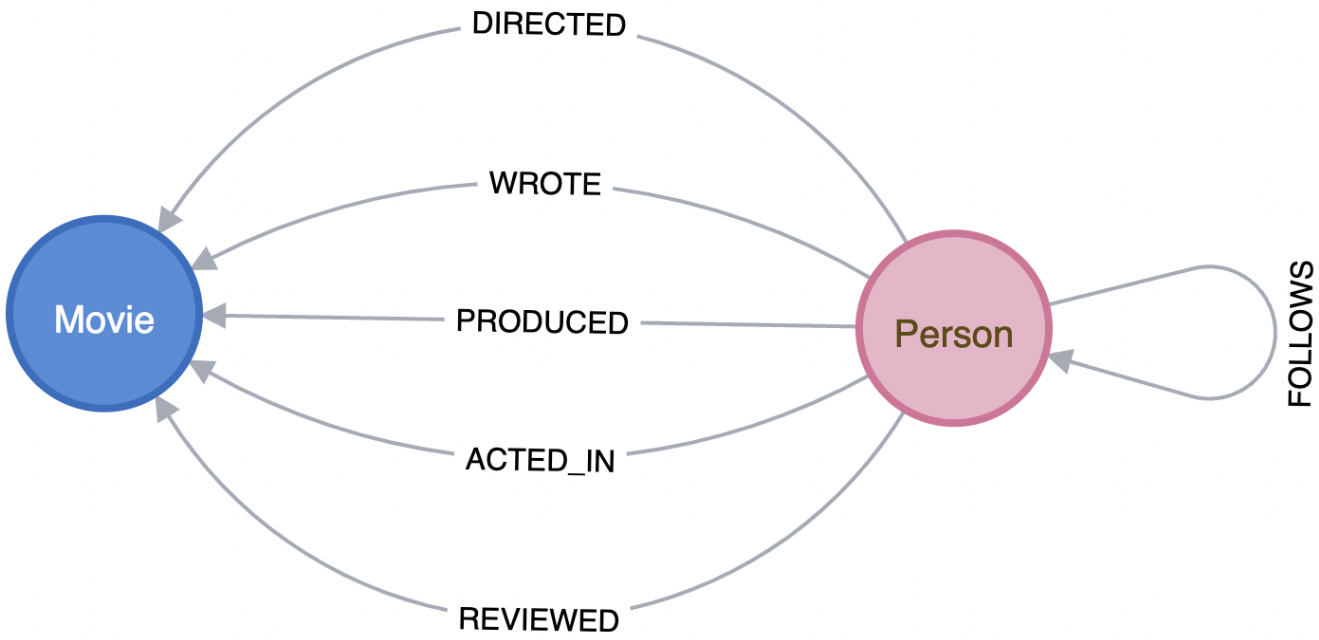


Figure 2: Graph Database Example from the Neo4J documentation.

# 3 Enhancing the Todo App with Storage

Last week we created a simple web server that can listen on a port and respond to HTTP requests. The endpoints that we created are all stubs that return a hard coded JSON response. This week we will add a database to our application to support persistent storage.

## 3.1 Creating a Practical Repository

Navigate to the GitHub Classroom link for this practical provided on the EdStem discussion board. As with last week, this will create a new repository for you in the course organisation. Clone this repository to your local machine or work directly in the browser with GitHub Codespaces. This repository is populated with our solution to last week's practical exercise. You may modify this solution or replace it with your own.

## 3.2 Installing the Database Dependencies

We will be using a Python library called SQLAlchemy to interact with our database. This library abstracts the SQL queries away and allows us to interact with the database using Python objects. We will be using SQLAlchemy via a package called Flask-SQLAlchemy, a wrapper around SQLAlchemy that is designed to work with Flask, our web server library.

```
$ poetry add flask-sqlalchemy
```

## 3.3 Creating the Database and Models

We will be using a database called SQLite for this practical. SQLite is a file-based database which is easy to setup and use. As the database is isolated to a file, SQLite is a suitable choice for initial development.

Now that our dependencies are installed, navigate to the cloned practical directory and create a new folder within the `todo` folder called `models`. Inside this folder create a new file called `todo.py` and a new file called `__init__.py`.

Inside the `__init__.py` file add the following code.

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

All this file does is setup a new SQLAlchemy object which we will use to interact with our database. In the `models/todo.py` file add the following code.

```
import datetime
from . import db

class Todo(db.Model):
    __tablename__ = 'todos'

    # This is how we define a column. It is also the primary key.
    id = db.Column(db.Integer, primary_key=True)
    # This is a manadatory column of 80 characters.
    title = db.Column(db.String(80), nullable=False)
    # This is an optional column of 120 characters.
    description = db.Column(db.String(120), nullable=True)
    # This column has a default value of False.
    completed = db.Column(db.Boolean, nullable=False, default=False)
    deadline_at = db.Column(db.DateTime, nullable=True)
    # This column has a default value which is a function call.
    created_at = db.Column(db.DateTime, nullable=False, default=datetime.datetime.now
        )
    # This column has a default value which is a function call.
    # It also updates on update.
    updated_at = db.Column(db.DateTime, nullable=False,
                           default=datetime.datetime.now,
                           onupdate=datetime.datetime.now)

    # This is a helper method to convert the model to a dictionary.
    def to_dict(self):
        return {
            'id': self.id,
            'title': self.title,
            'description': self.description,
            'completed': self.completed,
            'deadline_at': self.deadline_at.isoformat() if self.deadline_at else None,
            'created_at': self.created_at.isoformat() if self.created_at else None,
            'updated_at': self.updated_at.isoformat() if self.updated_at else None,
        }

    def __repr__(self):
```

```
        return f'<Todo {self.id} {self.title}>'
```

The above code is doing a lot of the heavy lifting for us in our database table generation. Look at the comments in the code to see what each line is doing.

## 3.4 Configuring the Database

Now that we have defined our database schema using an ORM, we need to configure our application to use the database. Open the `todo/__init__.py` file and change the code to the following.

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

def create_app():
    app = Flask(__name__)

    app.config['SQLALCHEMY_DATABASE_URI'] = "sqlite:///db.sqlite"

    # Load the models.
    from todo.models import db
    from todo.models.todo import Todo
    db.init_app(app)

    # Create the database tables.
    with app.app_context():
        db.create_all()
        db.session.commit()

    # Register the blueprints.
    from todo.views.routes import api
    app.register_blueprint(api)

    return app
```

The code above sets a default location, defined as a URI, for our database. As we using SQLite we can set the database to be a file in the file system. In production systems, you would set this to a URI that defines the credentials and hostname of your database server. In latter practicals, we will get experience using more complex URIs.

If we run our application now, we will see that the database file has been created for us.

```
$ poetry run flask --app todo run -p 6400 --debug
```

Your project structure should at least have the follow structure (there may be additional files).

```
/
├── instance
│   └── db.sqlite
├── tests
│   └── test_todo.py
├── todo
│   ├── models
│   │   ├── __init__.py
│   │   └── todo.py
│   ├── views
│   │   └── routes.py
│   └── __init__.py
├── .gitignore
├── endpoints.http
├── poetry.lock
├── pyproject.toml
└── README.md
```

## 3.5 Inspecting the Database

**Info**

Most platforms including MacOS, WSL2, and most Linux environments, come with SQLite installed. If you get an error running the commands below, you may have to install SQLite.

We can use the `sqlite3` command line tool to inspect the database. Open a terminal and navigate to the root of your project. Then run the following command.

```
$ sqlite3 instance/db.sqlite
```

This will open the SQLite command line tool and connect to the database file. We can then run the following command to see the tables in our database.

```
.tables
```

This will show us the tables in our database. We can then run the following command to see the schema of our table.

```
.schema todos
```

This will show us the columns in our table. We can then run the following command to see the data in our table.

```
SELECT * FROM todos;
```

This should initially show us no output, indicating that the table is currently empty. We can then run the following command to exit the SQLite command line tool.

```
.exit
```

You should notice that our table is called todos and not todo because we specified todos with the `__tablename__` attribute.

## 3.6 Using the Database

Now that we have a database integrated into our application, we will modify our endpoints to take advantage of it.

Open the `todo/views/routes.py` file and add the following imports to the top of the file.

```
from flask import Blueprint, jsonify, request
from todo.models import db
from todo.models.todo import Todo
from datetime import datetime
```

Now change the `get_todos` endpoint to the following.

```
@api.route('/todos', methods=['GET'])
def get_todos():
    todos = Todo.query.all()
    result = []
    for todo in todos:
        result.append(todo.to_dict())
    return jsonify(result)
```

This will query the database for all the todos and return them as JSON. We can then change the `get_todo` endpoint to the following.

```
@api.route('/todos/<int:todo_id>', methods=['GET'])
def get_todo(todo_id):
    todo = db.session.get(Todo, todo_id)
    if todo is None:
        return jsonify({'error': 'Todo not found'}), 404
    return jsonify(todo.to_dict())
```

Now we have modified these endpoints to use the database — let's test that our application is still functioning correctly. Restart your web server and navigate to the `/api/v1/todos` endpoint. You should see the following JSON response.

```
[]
```

Of course our application does not have any todo items in it yet. We will now add the ability to insert todo items into our database. In the `todo/views/routes.py` file, change the `create_todo` endpoint to the following.

```
@api.route('/todos', methods=['POST'])
def create_todo():
    todo = Todo(
        title = request.json.get('title'),
        description = request.json.get('description'),
        completed = request.json.get('completed', False),
    )
    if 'deadline_at' in request.json:
        todo.deadline_at = datetime.fromisoformat(request.json.get('deadline_at'))

    # Adds a new record to the database or will update an existing record.
    db.session.add(todo)
    # Commits the changes to the database.
    # This must be called for the changes to be saved.
    db.session.commit()
    return jsonify(todo.to_dict()), 201
```

This endpoint now creates a todo item in the database. Test this endpoint by going to `endpoints.http` (or your API query tool of choice) and running the POST request. You should see the following response (with different created_at and updated_at values).

```
{
  "id": 1,
  "title": "Test Todo",
  "description": "This is a test todo",
  "completed": false,
  "deadline_at": null,
  "created_at": "2026-02-27T15:30:00.000000Z",
  "updated_at": "2026-02-27T15:30:00.000000Z"
}
```

Now if we go to our `/api/v1/todos` endpoint we should see the todo item we just created.

```
[
  {
    "id": 1,
    "title": "Test Todo",
    "description": "This is a test todo",
    "completed": false,
    "deadline_at": null,
    "created_at": "2026-02-27T15:30:00.000000Z",
    "updated_at": "2026-02-27T15:30:00.000000Z"
  }
]
```

Now let's update the remaining endpoints. Change the `update_todo` endpoint to the following.

```
@api.route('/todos/<int:todo_id>', methods=['PUT'])
def update_todo(todo_id):
    """Update a todo item and return the updated item"""
    todo = db.session.get(Todo, todo_id)
    if todo is None:
        return jsonify({'error': 'Todo not found'}), 404

    todo.title = request.json.get('title', todo.title)
    todo.description = request.json.get('description', todo.description)
    todo.completed = request.json.get('completed', todo.completed)
    todo.deadline_at = request.json.get('deadline_at', todo.deadline_at)
    db.session.commit()

    return jsonify(todo.to_dict())
```

This endpoint will update a todo item in the database. Let's test this endpoint by going to `endpoints.http` and running the PUT request. You should see a response like the following.

```
{
  "id": 1,
  "title": "Updated Test Todo",
  "description": "This is an updated test todo",
  "completed": false,
  "deadline_at": null,
  "created_at": "2026-02-27T15:30:00.000000Z",
  "updated_at": "2026-02-27T15:35:00.000000Z"
}
```

To implement delete functionality, we will use the `HTTP DELETE` method and the `delete` method of the database session. In the `todo/views/routes.py` file, change the `delete_todo` endpoint to the following.

```
@api.route('/todos/<int:todo_id>', methods=['DELETE'])
def delete_todo(todo_id):
    """Delete a todo item and return the deleted item"""
    todo = db.session.get(Todo, todo_id)
    if todo is None:
        return jsonify({}), 200

    db.session.delete(todo)
    db.session.commit()
    return jsonify(todo.to_dict()), 200
```

We now have a set of endpoints that can perform the CRUD[1] operations of our API but some functionality is missing. We are going to add that functionality after setting up some tests to help us do **Test Driven Development**[2].

[1] Create, Read, Update, Delete
[2] https://agilealliance.org/glossary/tdd/

# 4 Testing the API

## 4.1 Setting up the testing environment

In the project, you have a `tests` folder. This contains `test_todo.py` which has a range of provided tests for the todo endpoints. However, we need to setup a few components to make it work.

Inside the `tests` folder create a `base.py` file. Add the following code to `base.py`.

```
from todo import create_app
import unittest

class TodoTest(unittest.TestCase):
    def setUp(self):
        self.app = create_app(config_overrides={
            'SQLALCHEMY_DATABASE_URI': 'sqlite:///:memory:',
            'TESTING': True
        })

        self.client = self.app.test_client()

    def assertDictSubset(self, expected_subset: dict, whole: dict):
        for key, value in expected_subset.items():
            self.assertEqual(whole[key], value)
```

This base class is what we will use to help setup our tests and provide an assertion helper method. The `setUp` method is called before each test and is used to initialise the in-memory database. The `assertDictSubset` method is a helper method that we will use to compare the todo items we actually get from the API with the todo items we expect to get from the API.

As you can see, we use a slight modification to the `create_app` function. We are passing in a dictionary of config overrides. This allows us to override the config values for the testing environment.

## 4.2 Prepping the config for testing

Open the `todo/__init__.py` file and update the `create_app` function to be as follows.

```
def create_app(config_overrides=None):
    app = Flask(__name__)

    app.config['SQLALCHEMY_DATABASE_URI'] = "sqlite:///db.sqlite"
    if config_overrides:
        app.config.update(config_overrides)

    # Load the models.
    from todo.models import db
    from todo.models.todo import Todo
    db.init_app(app)

    # Create the database tables.
    with app.app_context():
```

```
        db.create_all()
        db.session.commit()

    # Register the blueprints.
    from .views.routes import api
    app.register_blueprint(api)

    return app
```

## 4.3 Writing our first tests

Now we will write our first tests. In the `tests` folder and create a `test_health.py` file. Add the following code.

```
from tests.base import TodoTest

class TestHealth(TodoTest):
    def test_health(self):
        response = self.client.get('/api/v1/health')
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.json, {'status': 'ok'})
```

This test will make a GET request to the `/api/v1/health` endpoint and check that the response is a

- 200 status code, and
- JSON object with the key `status` and the value `ok`.

To run this test stop your server, if it is running, and run the following command.

```
$ poetry run python3 -m unittest tests/test_health.py
```

You should see the following output.

```
>> poetry run python3 -m unittest tests/test_health.py
....
----------------------------------------------------------------------
Ran 1 tests in 0.011s

OK
```

## 4.4 Test Driven Development

You have created a test for the health endpoint and run it. To run the suite of provided tests, use the following command.

```
$ poetry run python3 -m unittest discover -s tests
```

If you have used only the code we have provided, you should see some of the tests fail.

The tests are based on the specification from week one. We would like you to change your API to make the tests pass.

**Hint**

To retrieve the value of a request parameter, use `request.args.get()`[a]. To check if the request is a JSON request, you can use the `request.is_json`[b] attribute.

[a] https://tedboy.github.io/flask/interface_api.incoming_request_data.html#flask.Request.args
[b] https://tedboy.github.io/flask/interface_api.incoming_request_data.html#flask.Request.is_json

If you get stuck feel free to ask your peers / staff for help.

# 5 Finishing Up

We now have a working API which we can use to create, read, update, and delete todo items. We can also use the API to

- mark todo items as completed;
- filter todo items by whether they are completed or not, using the `completed` query parameter in the list get query); and
- filter todo items by whether they are within the next *N* days, using the `window` query parameter.

Next week we will Dockerise our API and use `docker compose` to run our API and a database within containers.

# References

[1] M. Kleppmann, *Designing Data-Intensive Applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media, Inc., March 2017.